5. Generate text using a pre-trained Large Language Model and analyze the generated responses.

Aim:

To generate text using a pre-trained large language model and analyze its responses.

Algorithm:

Start the program.
Import `AutoTokenizer` and `AutoModelForCausalLM` from the Transformers library.
Load the GPT-2 tokenizer and pre-trained language model.
Define an input prompt for text generation.
Convert the prompt into token IDs.
Pass the token IDs to the GPT-2 model.
Generate additional tokens using the model's text-generation method.
Decode the generated token IDs into readable text.
Display the input prompt and generated response.
Analyze the response for relevance, coherence, fluency, and factual limitations.
Stop the program.

Code:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

# Load the GPT-2 language model
gpt_tokenizer = AutoTokenizer.from_pretrained("gpt2")

gpt_model = AutoModelForCausalLM.from_pretrained("gpt2")

# Input prompt
prompt = "Artificial intelligence can help students by"

# Convert prompt to token IDs
inputs = gpt_tokenizer(
    prompt,
    return_tensors="pt"
)

# Generate text
with torch.no_grad():
    output_ids = gpt_model.generate(
        **inputs,
        max_new_tokens=40,
        do_sample=False,
        pad_token_id=gpt_tokenizer.eos_token_id
    )

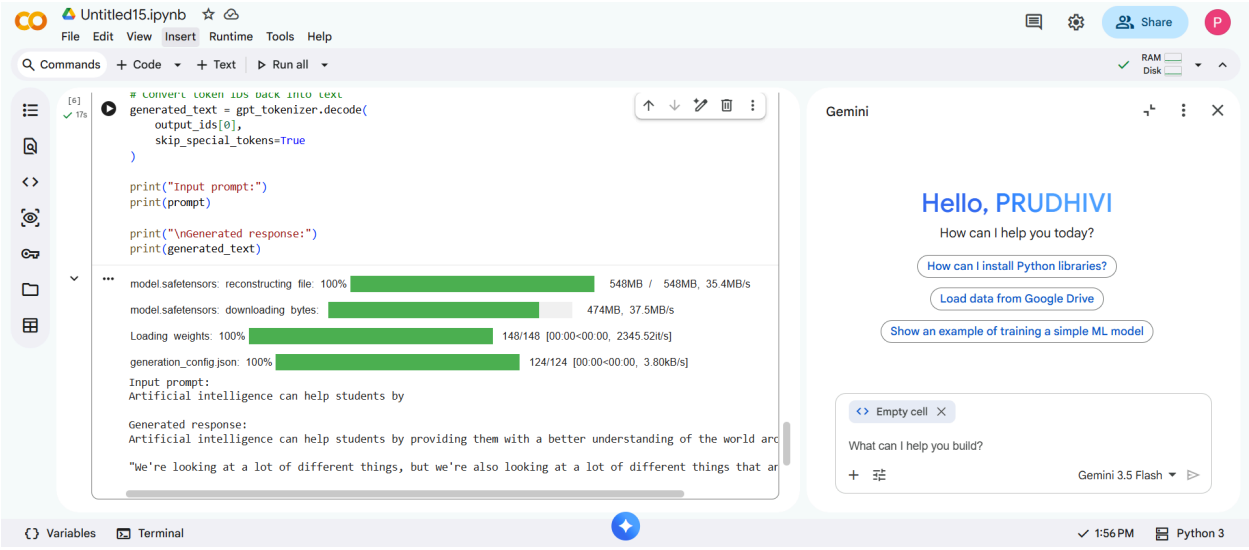
# Convert token IDs back into text
generated_text = gpt_tokenizer.decode(
```

```
    output_ids[0],
    skip_special_tokens=True
)

print("Input prompt:")
print(prompt)

print("\nGenerated response:")
print(generated_text)
```

Screenshot:



Result:

GPT-2 generated a text continuation from the given prompt. Its relevance, coherence, fluency, and limitations were examined.